CleanEater

By: Matej Pechoucek, Cieran Norrie, Razvan Arhip, Tarik Emre Sisman

Table of Contents

Introduction 2
Intended User Base: 2
General Users (Clients) 2
Professionals (Advisors and Contributors) 2
Functionality 3
Food and Nutrition Database 3
Example Scenario 3
Personalized Plan 4
Example Scenario 4
Nutrition Feedback System 5
Improvement System 6
Example Scenario 6
Framework 7
1. SQL (Database Management) 7
2. Flutter (Dart) 7
3. Recommandation Logic (Nutritional Proximity Algorithm) 7
Prototype Diagrams 7
Ethical Analysis 8
References 10
Appendix 11
Appendix 1: 11
Appendix 2: 12
Appendix 3: 13
Appendix 4: 14

Introduction

Maintaining a healthy diet and lifestyle in modern days is often a challenge. From misinformation on social media to the incentives of fast food everywhere you look. Any commitment to dieting often results in falling down the capitalist rabbit hole and becoming a victim of an unhealthy life. While medical professionals and nutritionists provide vital and professional expertise, their reach is generally limited by the time-intensive nature of consultations and their local area, where they may provide their wisdom. There is a significant gap between professional advice and the daily practical implications of that advice for the average person.

This is why the proposed solution, CleanEater, is a digital app designed to bridge the gap between professional and daily advice, providing structured guidance based on professional insights. By utilizing a strict database of accurate nutritional information and user goals/aims for the future, it can provide advice for most situations on daily dietary habits. Unlike general 'calorie trackers', Clean Eater provides feedback and suggestions for the future, tailored to the users' specified goals, such as improving blood results, or attempting to safely lose/gain weight.

Intended User Base:

General Users (Clients)

The primary demographic for CleanEater is anyone who wants to take control of their nutrition and habits without needing a degree in dietetics. This includes:

- Busy professionals and students: People who don't have time to meal prep perfectly and need quick, data-backed, and accurate decisions on the go.
- Fitness enthusiasts: Users looking to squeeze as much out of their daily nutrition goals to track specific macronutrients for improvement in and out of the gym.
- Health-conscious individuals: People trying to fix specific health markers or change their lifestyle sustainably, rather than just crash dieting or giving up.

Professionals (Advisors and Contributors)

This group includes certified nutritionists, dietitians, and fitness coaches who act as the human helpers within the app.

- Expert Q&A: They handle specific, complex questions in the “Question/Answer” system that the algorithm cannot solve – giving users peace of mind that their advice is safe and interpreted by a human instead of AI.
- Database Verification: They help us review user-submitted foods to keep nutrition data accurate for everyone and update our guidelines and standards with professional, dietetic research.
- Client Monitoring: It allows them to track any clients’ progress, seeing exactly what they eat and how much, without relying on food diaries or photos.

Functionality

Food and Nutrition Database

The Database is an essential part of the application that must satisfy the following requirements:

- Has a variety of commercially available foods with calorie and nutrient information
- Can synchronize with the cloud
- Easy steps to create new items with their specific information.
- Should link to healthier alternatives for foods considered unhealthy.

Example Scenario

This example demonstrates how the food database supports the core design requirements of the application as well as the user input in forms of scanning a barcode.

When a user scans a barcode for a specific brand of Greek Yogurt, the database instantly pulls data from a library of over 500,000 commercially available items. The system displays 150 calories, 15g of protein, and 0g of fat. It also includes micronutrients like Calcium and Vitamin B12, ensuring her daily "dashboard" is accurate.

The application is designed to synchronize user data through the cloud. Therefore, when the user logs their lunch while at work, the database pushes this entry to the cloud. Later, when the user opens another device at home to plan dinner, their "Calories Remaining" has already been updated. This design ensures ease of access across multiple devices and protects user data from device loss, replacement, or theft.

Now, a different user buys a "Local Artisan Honey" from a farmer's market that isn't in the system yet. The user can select to “Create a new item,” where they enter the name, serving size, and the three main macros from the label. The database saves this to their Personal Library and, as the user decides, flags it for "Global Review" so other users can potentially

benefit from their entry. To maintain database integrity while still allowing crowdsourcing, the system will implement a Tiered Trust Model:

- Tier 1 (Verified): Locked official data (USDA/Branded).
- Tier 2 (Community): User entries verified by 10+ independent users.
- Tier 3 (Personal): Visible only to the creator until verified, preventing database pollution.

Lastly, the user searches for "Potato Chips" to add to their snack log. The database recognizes "Potato Chips" as a high-sodium, high-fat item. Before the user logs the snack in, a "Healthier Choice" banner appears: "Looking for an alternative? Try Air-Potted Popcorn (saves 120 calories) or Baked Kale Chips (higher in Vitamin K)." This acts as a 'Smart-Swap' and healthier alternative for some foods/snacks.

Personalized Plan

While there is a general basis for the number of macronutrients, micronutrients, and general calories an adult should consume every day. However, this varies with several factors: age, weight, height, biological sex, activity level, and specific health goals (i.e., muscle gain, weight loss, or management of deficiencies).

Our tool moves away from the "one size fits all" approach, allowing each user to create their own specific program that aims to reach their goal. Instead of providing static numbers, the system calculates the Total Daily Energy Expenditure (TDEE) and then adjusts the macronutrient distribution based on the user's objective.

- Protein: Adjusted depending on lean body mass and activity intensity to ensure muscle growth/control.
- Fats: Calculated to support hormonal health, also ensuring a safe required total caloric intake.
- Carbohydrates: Function as the remaining sliding scale to fill in the remaining energy requirements based on energy demands specified by the user.

Example Scenario

A user aiming for "Healthy Weight Loss" will receive a plan that creates a sustainable and healthy 10-15% caloric deficit while increasing protein intake, which ensures satiety and metabolic health. Rather than suggesting to "eat less", the program takes a dynamic approach to allow the user to enjoy foods throughout the day and still be able to achieve their goal.

Technical Details: The system will implement the Mifflin-St Jeor Equation to calculate the BMR (Basal Metabolic Rate) due to its high accuracy in modern clinical settings. However, if the user wants to override a suggested BMR with which they may have previous experience, they are free to do so.

BMR = (10 x weight(kg)) + (6.25 x height(cm)) – (5 x age(years)) + s

(Where s is +5 for males and -161 for females)

The result will be multiplied by an activity factor (1.2 for sedentary to 1.9 for extra active users) to produce a daily target. Again, any personal objections or previous knowledge that could incentivize the user to adjust their ideal caloric intake accordingly.

Nutrition Feedback System

The program should utilize the database of food and nutrition information to provide alternative food ideas to the user that are more nutritious.

This will act as short-term advice and intervention in comparison to the personalized plan. Where the program will look at the recorded food and nutrient intake of the user over the past 3 weeks to analyze recent changes - a preventative measure to help users keep focus on their goals.

Example: The user has entered the following goals in their plan:

- Eat no more than 2500 calories per day.
- Eat 130g of protein per day with 40g of fiber per day.
- Eat no more than 100g of sugar per day.

With these scores from this week:

- Avg. 2450 calories per day
- Avg. 85g protein per day
- Avg. 140g sugar per day

The program may notice that the user has been eating cheesecake this week and so may recommend a fruit bowl with yogurt - the user still eats sweet food, but overall would eat less sugar.

The program may notice that the user has been eating less protein this week, and so may recommend recipes that are protein-heavy, such as salmon.

Instead of high-latency file processing, the app will utilize PostgreSQL window functions to instantly analyze intake trends. By executing a simple query, the system calculates rolling average and data in the backend database. This manner allows near immediate and secure

feedback without performance bottlenecks of CSV file parsing or data transfer. Analysis will be used to conclude if the data deviates from preset thresholds (the calorie / nutritional goals of the user).

If so, then the program will provide recommendations to help the user achieve their goals again.

Improvement System

The program will track user-inputted data and provide accurate and easy-to-understand graphs regarding the user's improvement towards their goals and objectives, what they have achieved in the past, and a potential section that shows what they can achieve if they continue to improve. The program will track the data that has been collected over the past data. A basic Linear Regression system will be used to calculate the slope of the deficit trend and predict a specific "Goal Achievement Date," to provide outlook and motivation based on adherence to the plan.

The span of collected data will get longer depending on how long the user has been using the app for and how long they want the span to be, by default it is one week long at the start and then the span progressively increases as the user keep using the app long enough for the data to be gathered, which caps at 6 weeks. But the user can change this feature so that the graphs will always show the last week, the last 2 weeks, the last 3 weeks, etc.

Example Scenario

The user has logged their performance for the past week every day, which consists of numerical scores such as calorie intake, protein intake, and sugar intake. The program would then plot these scores in conjunction with their goals in a way that's easily understandable to the user in the form of a bar graph.

The program will also analyze the graph and tell the user if they are achieving their goal, give tips depending on their current performance (In this case if they are undereating, overeating or are hitting their calorie goal) and show the user what they will achieve if they keep on maintaining their performance (For example if they have currently been hitting their calorie goals, the program will tell the user that if they keep on maintaining this performance, they could get down to their goal weight in x days).

The program will also analyze inputs hourly and show the user how many calories they have left, how they can fill those calories, and it will also count how many days the user has been hitting their goals and have a streak system that increases a counter depending on the user's performance. Appendix 1 contains an example diagram of the calorie summary.

Framework

1. SQL (Database Management)

We will use SQL to manage and interact with the fields and records of data stored within the database, so the software can respond with accuracy and recognize meaningful patterns within the database.

2. Flutter (Dart)

We will use Dart to build most of the application, as Dart tends to have a shorter learning curve compared to other languages that could be chosen (its syntax is similar to Java). Flutter is a UI toolkit[1] and allows for the implementation of customized, adaptive interfaces for our application to display well on mobile and on a computer.

Flutter also compiles to JavaScript, which would allow our software to have faster performance.

3. Recommandation Logic (Nutritional Proximity Algorithm)

To provide realistic alternatives without manual data entry, the prototype will utilize a Nutritional Proximity Algorithm based on Euclidian Distance[2]. The system will map every food item into a multi-dimensional vector space defined by its macronutrients (Protein, Fats, Carbohydrates), and some key micronutrients (Sugar, Sodium). When a user logs a 'unhealthy' food, (e.g., fried potato chips), the algorithm searches for neighboring foods using the Euclidian distance formula that share a similar nutritional profile. This means that a log of 'potato chips' with high sodium and high fat, may encourage an alternative of a food/snack including a specific health advantage (i.e., 50% less saturated fat or 25% less sodium). This ensures that suggestions satisfy the user's likely cravings rather that only offering an unrelated, unappealing healthy alternative which may be found 'annoying' and 'useless'.

Prototype Diagrams

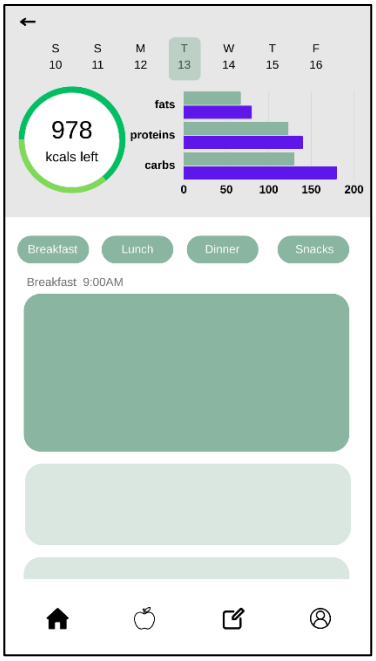
Prototype diagrams of the User Interface include the following pages: Home Page, Diet Planning Page, Question/Answer Page, and Profile Page. Each page has its own purpose, the home page acting as the default landing page where an overview of remaining calories will be shown, an overview of macronutrient intake, as well as a brief log and description of the

---

[1] "Flutter", URL: https://flutter.dev/ accessed: 13/02/2026

[2] "Wikipedia", URL: https://en.wikipedia.org/wiki/Proximity_analysis accessed: 13/02/2026

meals/foods that have already been logged. Alongside these features, a small calendar will be implemented where the user can either add logs to the future, for planning or ease of use, or go back in time to check their past logs and calorie goals for comparison. The diet planning page allows for more detailed caloric target planner. The question/answer page is for questions and answers, including most popular FAQs as well as expert inquiries. Lastly, the profile page shows more detailed diagrams and information of your trend and goals. Here is the main page for reference, while the others will be added to the appendix:



Main page prototype design made in Canva[3]

Ethical Analysis

Regarding the dieting advice, we must ensure that the algorithm is not too strict on the user's nutrition or calorie goals, as some deviation from a person's typical diet is not only normal but healthy in some cases. Specifically, we must avoid the algorithm always trying to get the user to eat the "perfect" food or diet to meet their goals. This could potentially damage the user's relationship with the software and with their food, which could lead to eating disorders and poor self-esteem. The user's data (their recent calorie/nutrition intake and foods eaten) must not be kept on their record for longer than necessary for the functioning of the software to ensure data confidentiality and user privacy, as dieting and food can be a very personal subject. In the case of a user adding a new food item to the software database,

[3] "Canva", URL: https://www.canva.com/ accessed 13/02/2026

there should be no links between the food item added and the user who added it (or any other user).

Additionally, although it's impossible to have a database of all foods and meals in the world, we must aim to ensure our dataset of food items are as culturally diverse as possible, as many foods from all over the world could help users to meet their goals, and users may feel more inclined to eat certain foods than others based on their experiences and cultural upbringing. Where this pledge is not realistic to maintain, we must be completely transparent as to what foods are in our database, so users are aware of whether their food preferences are represented in our software or not.

Moreover, even with accurate nutritional information, the system could be mistaken for medical advice, especially by users with existing conditions (i.e., diabetes, eating disorders, kidney disease). Users may inadvertently follow advice that is inappropriate for their medical needs, and over-reliance on automated feedback could delay seeking professional help. Therefore, clear disclaimers must state that the application is not a medical device and does not replace professional diagnosis. Furthermore, prompting users to consult professionals when patterns indicate potential health risks and warning users about high-risk goals will further mitigate risks.

However, to prevent obsessive behaviors, the app will use non-judgmental UI patterns where caloric surpluses are shown in neutral colors, and not in "danger red" or other "fear/stress-inducing" colors. The app will also prioritize weekly average rather than daily perfection to reduce psychological stress of potential outliers or 'cheat' days.

Another ethical concern is the potential reinforcement of unhealthy or disordered eating behaviors. Constant tracking of calories, nutrients, and performance metrics may encourage over-obsessive monitoring and guilt around food. To mitigate this, the system should avoid punitive language, allow flexibility in goal adherence, and frame feedback in a supportive and non-judgmental manner. Features such as streaks and progress indicators should be optional and designed to promote long-term health and well-being, rather than perfection or worry of 'not reaching your goals'.

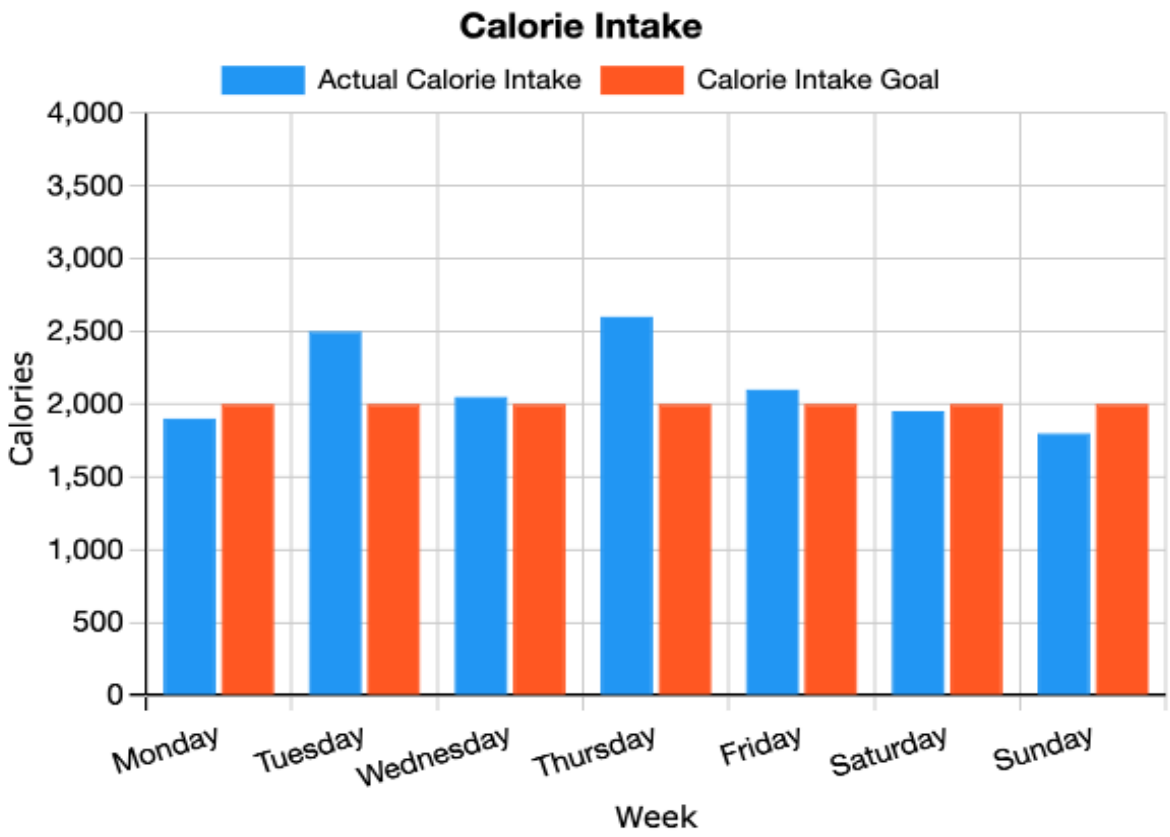
References

[1] “Flutter”, URL: https://flutter.dev/ accessed: 13/02/2026

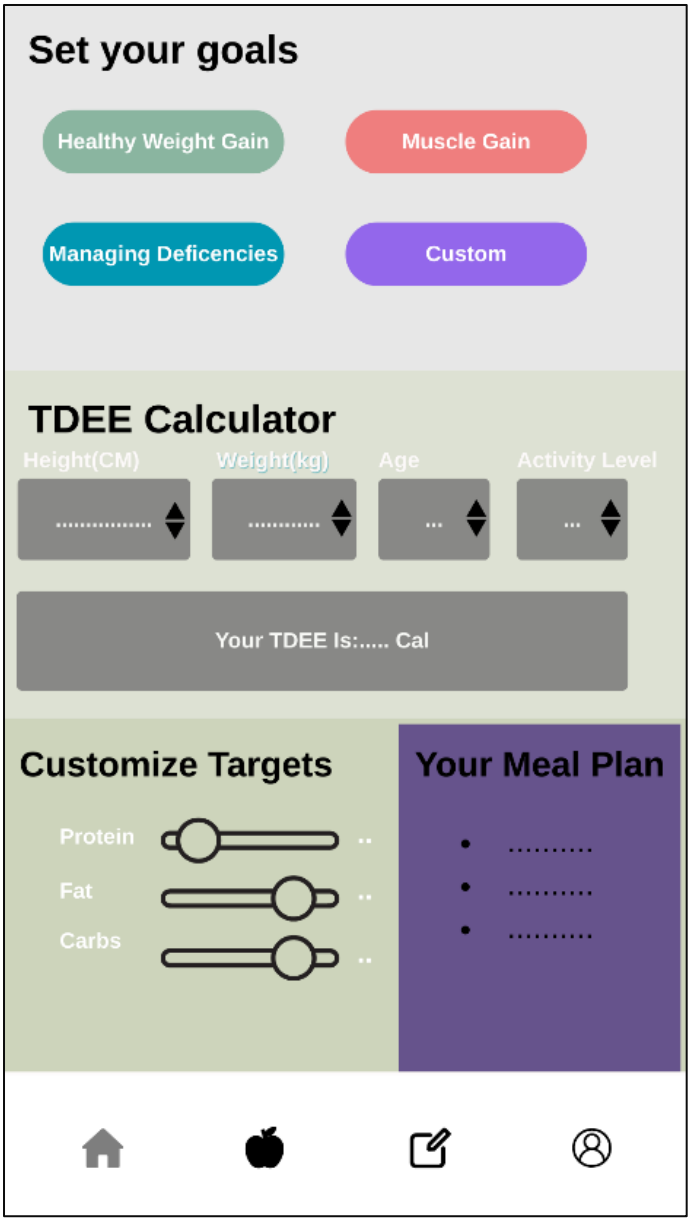
[2] “Wikipedia”, URL: https://en.wikipedia.org/wiki/Proximity_analysis accessed: 13/02/2026

[3] “Canva”, URL: https://www.canva.com/ accessed 13/02/2026

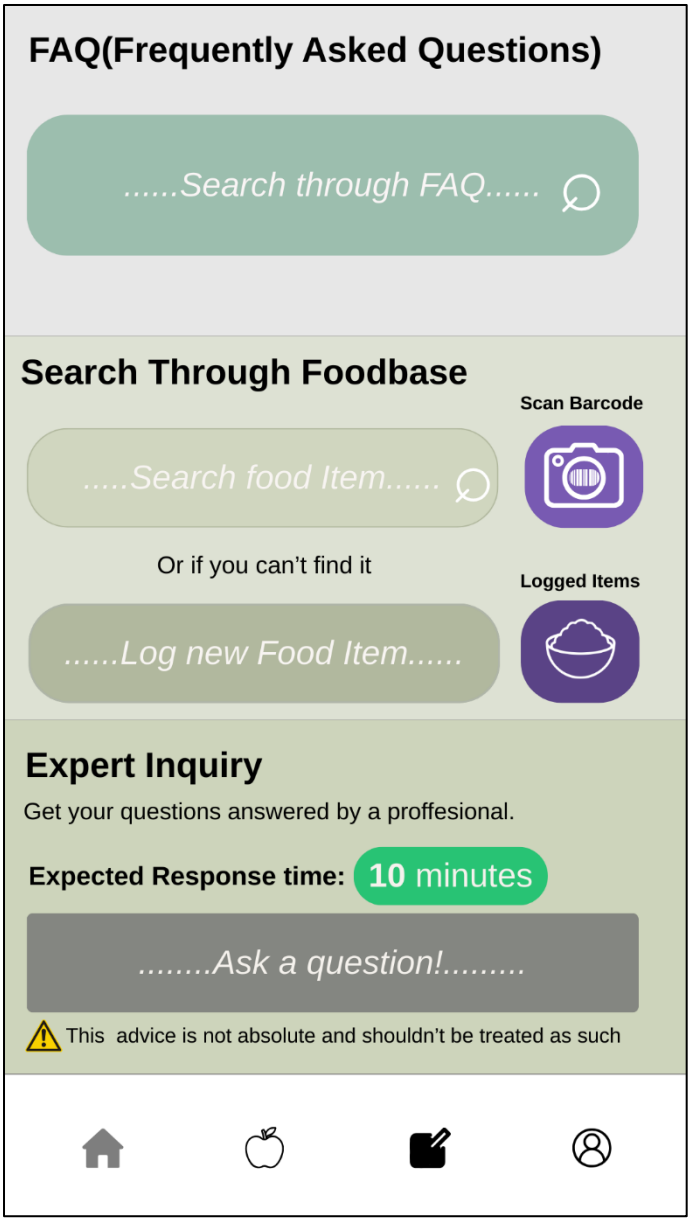
Appendix



Appendix 1: Example of a diagram of the calorie summary in a day for a user.



Appendix 2: Diet Planning Page prototype UI design made in Canva.



Appendix 3: Question/Answer Page prototype UI design made in Canva.

Appendix 4: Profile/Diagram Page prototype UI design made in Canva.